# Quantum Computing *without* the Linear Algebra

Aws Albarghouthi
University of Wisconsin–Madison

**Abstract**

Quantum computing is often introduced through the lens of linear algebra with notation that is inherited from quantum mechanics. In this paper, we take an operational view of quantum computing that is easy to demonstrate programmatically. The hope is that this viewpoint will (1) demystify quantum computing and make it more accessible to a wider audience, particularly computer science students and software engineers, and (2) possibly serve as the basis of a formal foundation for automatically reasoning about quantum programs.

We treat the state of a quantum computer as a set and present the operations of a quantum computer—quantum gates and measurements—using familiar functional set transformations (think: map, filter, fold, etc.). By the end of the paper, we will have implemented a simple quantum circuit simulator that can be used to simulate small quantum circuits. The code is available at https://github.com/qqq-wisc/qwla.

## 1 Introduction

Quantum computing offers an exciting new way to build computers by harnessing the strange properties of quantum mechanics. The idea emerged in the 1980s when scientists realized they needed a better way to simulate nature at its tiniest scales. Today, research labs are making steady progress toward building practical quantum computers. These machines promise to solve important problems that regular computers find impossibly hard—like breaking encryption codes by factoring huge numbers or modeling new materials and medicines by simulating their quantum behavior. While still an emerging technology, quantum computing *may* revolutionize how we process information and unlock new frontiers in science and technology.

Because quantum computing is a consequence of quantum mechanics, it is introduced through the lens of linear algebra with notation that is inherited

from physics. While the linear-algebraic view is a powerful theoretical foundation, it often complicates and obscures the simple mechanics of quantum computation. My goal in this paper is to present a different view of quantum computing that is hopefully more accessible to computer scientists and software engineers.

Specifically, I will present a way of thinking about quantum computing that is based on the standard set data structure. Then, the instructions of a quantum computer end up being simple set transformations—map, filter, reduce, and the like—that are available in all mainstream programming languages, from Python to Java to Rust. I think this view can help demystify quantum computing, making it more accessible to a wider audience, particularly those who are not already steeped in linear algebra. From a research perspective, this functional view can potentially serve as a formal foundation for automatically reasoning about quantum programs.

The paper goes step by step, starting with the definition of a quantum state as a set and then describing the operations of a quantum computer, after which we will have all we need to implement a full-blown quantum-program simulator. Finally, we will look at a few simple quantum algorithms and visualize them using our approach.

## 2 Quantum States

In computing, we think of the state of a program as a sequence of bits. With each program instruction, we flip some of those bits. In quantum computing, instead of manipulating bits, we manipulate *qubits*. A qubit is just like a bit: it can be $0$ or $1$, but it can also be a combination of both! Specifically, a qubit can be a $0$ with some weight, or *amplitude*, and a $1$ with some other amplitude. An amplitude is just a number—technically, a complex number, but for most of our presentation real numbers will suffice.

Because of this property of qubits, we can think of the state of a quantum program as a set of amplitudes, one for each possible qubit value. For example, if we have a single qubit, then we present it as the following set of *key–value* pairs (or dictionary):

$$\{(0, a), (1, b)\}$$

Each element of this set is a bitstring (the key) along with its amplitude ($a$ and $b$ in this case, the values). We say that this qubit is $0$ with amplitude $a$ and $1$ with amplitude $b$. For clarity, we will write the set as follows, with the bitstrings on the top and the amplitudes on the bottom.

$$\begin{matrix} 0 & 1 \\ \{a & b\} \end{matrix}$$

In their simplest setting, qubits represent good old bits. The following examples show qubits that represent the bit-values $0$ and $1$. This is a qubit in state $0$ (notice that the amplitude of $0$ is $1$ and the amplitude of $1$ is $0$)

$$\begin{matrix} 0 & 1 \\ \{1 & 0\} \end{matrix}$$

and this is a qubit in state $1$:

$$\begin{matrix} 0 & 1 \\ \{0 & 1\} \end{matrix}$$

Now comes the fun part. The qubit can be in a combination of both states! This situation is known as a *superposition* of $0$ and $1$, for example:

$$\begin{matrix} 0 & 1 \\ \left\{\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}}\right\} \end{matrix}$$

It is also possible to have negative amplitudes, for example:

$$\begin{matrix} 0 & 1 \\ \left\{\frac{1}{\sqrt{2}} & \frac{-1}{\sqrt{2}}\right\} \end{matrix}$$

The squared absolute values of the amplitudes must add up to $1$. This is an important property because, as we shall see later, we will interpret the amplitudes as probabilities, and probabilities must sum to $1$. The property is true in the above example:

$$\left|\frac{1}{\sqrt{2}}\right|^2 + \left|\frac{-1}{\sqrt{2}}\right|^2 = 1.$$

Unlike with classical computers, we can't simply read an amplitude off a quantum computer's memory. Instead, the so-called qubit *measurement* process returns the state of a qubit probabilistically, proportional to the absolute value of the amplitude. In our two superposition examples above, the probability of reading $0$ is $\frac{1}{2}$ and the probability of reading $1$ is also $\frac{1}{2}$. We will talk about measurement in more detail later.

When we have more than one qubit, we maintain an amplitude for each possible combination of qubit values, that is, $2^n$ amplitudes for $n$ qubits. For example, if we have two qubits, we have four possible combinations: $00$, $01$, $10$, and $11$. The following quantum state is $01$:

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{0 & 1 & 0 & 0\} \end{matrix}$$

The following two-qubit state is an equal superposition of all four combinations:

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{\frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2}\} \end{matrix}$$

Note that $4\left(\frac{1}{2}\right)^2 = 1$.

This example illustrates the crux of quantum computing, the ability to represent—and, as we will see in a bit, manipulate—exponentially many states simultaneously. However, making use of this power is not as straightforward as it may seem. More on this later.

### Implementing Quantum States

Throughout this paper, we will be manipulating quantum states. We represent the states just as in the diagrams above, as a set of key–value pairs, where each bitstring is associated with an amplitude (a complex number). So, in general, a quantum state with $n$ qubits is a set of the form:

$$\{(b^1, a^1), (b^2, a^2), \ldots, (b^{2^n}, a^{2^n})\}$$

where $b^j$ is a bitstring and $a^j$ is its amplitude.

In what follows, we will use pseudocode for defining operations over the state. All of our code is set transformations in the form of map, reduce, filter, etc. For example, the following piece of code removes all elements with zero amplitude in a quantum state $s$:

$$s.\mathsf{filter}(\lambda b, a.\ a \neq 0)$$

where the arguments of the lambda function are each bitstring–amplitude pair, $(b, a)$, in the state $s$. So if $s = \{(0,1),(1,0)\}$, then $s.\mathsf{filter}(\lambda b, a.\ a \neq 0)$ is $\{(0,1)\}$.

The following code, on the other hand, flips the signs of the amplitudes:

$$s.\mathsf{map}(\lambda b, a.\ (b, -a))$$

So, if $s = \{(0,1),(1,0)\}$, then $s.\mathsf{map}(\lambda b, a.\ (b, -a))$ is $\{(0,-1),(1,0)\}$.

For bitstrings $b$, we will use the notation $b_j$ to refer to the $j$th bit in $b$, with $b_0$ being the leftmost bit. The accompanying implementation is written in Python and very much resembles the pseudocode, but the pseudocode elides language idiosyncrasies.

## 3 Quantum Operations

Now that we have a way to represent a quantum state, we can start thinking about how to manipulate it using quantum operations—which are also called

gates, because people typically think of quantum programs as circuits. An operation on a quantum state should satisfy a few properties dictated by the laws of quantum mechanics:

- *Reversibility*: It should be reversible. I can't, say, randomly shuffle the amplitudes or zero them out, because then I would not be able to recover the initial state.
- *Normalization preservation*: It should preserve the fact that the squares of the absolute values of the amplitudes sum to $1$.
- *Linearity*: As we will see, you should be able to write any quantum operation as a map (and sometimes reduce) over the state—remember, the quantum state is a set of pairs! In fact, most quantum operations can be written in the form:

  $$s.\mathsf{map}(\lambda b, a.\ (b', ca))$$

  In other words, we simply transform each bitstring–amplitude pair $(b, a)$, independently, into a new pair $(b', ca)$, where the new bitstring $b'$ and the coefficient $c$ are functions of $b$ (not the amplitude $a$).

## The $X$ Gate

Let's consider one of the simplest gates, the NOT gate, or the $X$ gate as it is known in the literature. Intuitively, this gate swaps the amplitudes of $0$ and $1$. Namely, the element $(0, a)$ is transformed into $(1, a)$ and the element $(1, b)$ is transformed into $(0, b)$.

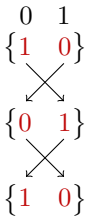
Here's an example of applying $X$ to the state $0$:

$$\begin{matrix} 0 & 1 \\ \{1 & 0\} \end{matrix}$$

$$\begin{matrix} 0 & 1 \\ \{0 & 1\} \end{matrix}$$

Notice how $(0, 1)$ is transformed into $(1, 1)$ and $(1, 0)$ is transformed into $(0, 0)$.

The $X$ gate is reversible; in fact, it is its own inverse. Apply $X$ again and you will swap the two amplitudes back:




Also note that, because $X$ doesn't modify the amplitudes, just swaps them, it preserves normalization.

Let's implement the $X$ gate. Recall that our state is represented as a set where each element is a pair of a bitstring and its amplitude. Say we want to apply $X$ to the $j$th qubit in a state $s$. All we have to do is transform each element of the state by flipping the $j$th bit in the bitstring. Programmatically, we map each element of the state to a new element, as follows:

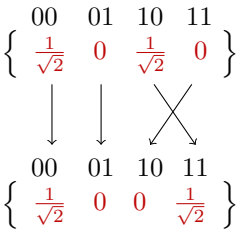
$$s.\mathsf{map}(\lambda b, a.\ (b^{\neg j}, a)) \qquad (X_j)$$

Here, $b^{\neg j}$ is $b$ but with the $j$th bit flipped.

## The $CX$ Gate

Now let's consider a more complex gate, the $CX$ gate. This is the quantum analog of a simple conditional operation. It operates over two qubits, the *control* and the *target*. If the control qubit is 1, then the target qubit is flipped. For example, if we have the element $(11, a)$, then the $CX$ gate, with the control qubit being the leftmost qubit, will transform it to $(10, a)$.

The following example illustrates the $CX$ gate assuming the control qubit is the leftmost qubit:




Notice that the amplitudes of 00 and 01 are unchanged, but the amplitudes of 10 and 11 are swapped, because the control qubit is 1.

Let's implement the $CX$ gate. Say we want to apply it to the $j$th (control) and $k$th (target) qubits in some state $s$:

$$s.\mathsf{map}(\lambda b, a.\ (ite(b_j, b^{\neg k}, b), a)) \qquad (CX_{j,k})$$

The if-then-else expression $ite(b_j, b^{\neg k}, b)$ flips the $k$th qubit in the bitstring $b$ if the $j$th qubit is 1, and otherwise leaves the bitstring unchanged.

## The $S$ and $T$ gates

The above gates all feel like they are classical—they did not touch the amplitudes, just moved them around. We will now look at two single-qubit gates that *modify* the amplitudes. Both gates we will see multiply the amplitude of the $1$ state by a complex number, leaving the amplitude of the $0$ state unchanged. Specifically, they will both multiply the amplitude of the $1$ state by a complex number $c$ where $|c| = 1$. This ensures that the normalization property is preserved.[1]

In principle, the complex number $c$ with which we multiply the amplitude could be any complex number such that $|c| = 1$, but to build a quantum computer, it suffices to only consider a couple of a concrete values, like the $S$ and $T$ gates described below:

**The $S$ gate** The $S$ gate applies to a single qubit and is defined as follows: If the state is $0$, then its amplitude remains unchanged. If the state is $1$, then its amplitude is multiplied by the imaginary unit $i$. For example:

$$\begin{array}{cc} 0 & 1 \\ \left\{ \begin{array}{cc} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{array} \right\} & \\ \downarrow \quad \downarrow & \\ 0 \quad 1 & \\ \left\{ \begin{array}{cc} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} i \end{array} \right\} & \end{array}$$

Again, observe that the $S$ gate is reversible and also preserves the property that the squared absolute values of the amplitudes sum to 1, that is,

$$\left|\frac{1}{\sqrt{2}}\right|^2 + \left|\frac{1}{\sqrt{2}} i\right|^2 = 1$$

It is instructive to also see how the $S$ gate looks when applied to the leftmost qubit of a 2-qubit state:

[1] Recall that a complex number is of the form $a + bi$, where $a$ and $b$ are real numbers, and $i$ is the imaginary unit. The absolute value of a complex number $a + bi$ is defined as $\sqrt{a^2 + b^2}$.

$$\begin{array}{cccc} 00 & 01 & 10 & 11 \\ \{\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2}\ \} \\ \downarrow & \downarrow & \downarrow & \downarrow \\ 00 & 01 & 10 & 11 \\ \{\ \frac{1}{2} & \frac{1}{2} & \frac{1}{2}i & \frac{1}{2}i\ \} \end{array}$$

Notice that only the amplitudes of the $1*$ bitstrings are multiplied by $i$; the amplitudes of the $0*$ bitstrings are left unchanged.

Let's implement the $S$ gate. Say we want to apply it to the $j$th qubit in a state $s$:

$$s.\mathsf{map}(\lambda b, a.\ (b, ai^{b_j})) \tag{$S_j$}$$

The expression $i^{b_j}$ is $1$ if $b_j = 0$ and $i$ if $b_j = 1$, thus ensuring that the amplitude of the bitstrings with $j$th bit set to $0$ is unchanged, while the amplitude of the bitstrings with $j$th qubit set to $1$ is multiplied by $i$.

**The $T$ Gate** The $T$ gate is similar to the $S$ gate, but it multiplies the amplitude by a $\frac{1+i}{\sqrt{2}}$ for the $1$ state, leaving the $0$ state unchanged, for example:[2]

$$\begin{array}{cc} 0 & 1 \\ \left\{\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}}\ \right\} \\ \downarrow & \downarrow \\ 0 & 1 \\ \left\{\ \frac{1}{\sqrt{2}} & \frac{1+i}{\sqrt{4}}\ \right\} \end{array}$$

We can implement the $T$ gate as follows, assuming we want to apply it to the $j$th qubit in $s$:

$$s.\mathsf{map}\left(\lambda b, a.\ \left(b, a\left(\frac{1+i}{\sqrt{2}}\right)^{b_j}\right)\right) \tag{$T_j$}$$

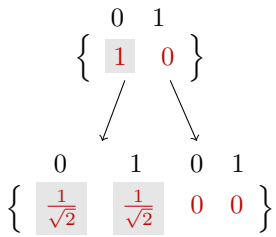
The expression $\left(\frac{1+i}{\sqrt{2}}\right)^{b_j}$ is $1$ if $b_j = 0$ and $\frac{1+i}{\sqrt{2}}$ if $b_j = 1$, thus ensuring that the amplitude of the bitstrings with $j$th qubit set to $0$ is unchanged, while the amplitude of the bitstrings with $j$th qubit set to $1$ is multiplied by $\frac{1+i}{\sqrt{2}}$.

[2]Note that $\left|\frac{1+i}{\sqrt{2}}\right|^2 = \sqrt{\frac{1}{2} + \frac{1}{2}} = 1$. Also note that $\frac{1+i}{\sqrt{2}}$ is the $\pi/4$ rotation of $i$ in the complex plane, which is typically written as $e^{i\pi/4}$.

## The Hadamard Gate

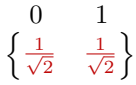
Finally, we arrive at the Hadamard gate, which is perhaps the most unusual of all the gates. It applies to a single qubit and puts it in superposition. In all of the gates we discussed so far, they mapped each element of the state to a new element. The Hadamard gate, however, will take an element and turn it into *two* elements. Superposition! For example, the element $(0, 1)$ is mapped to $(0, \frac{1}{\sqrt{2}})$ and $(1, \frac{1}{\sqrt{2}})$, while the element $(1, 1)$ is mapped to $(0, \frac{1}{\sqrt{2}})$ and $(1, -\frac{1}{\sqrt{2}})$ (notice the negative sign).

Let's look at a step-by-step example:

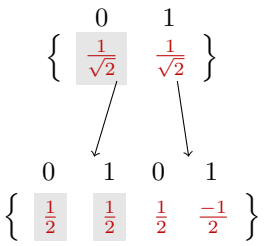


Here, each element of the state is mapped to two elements. Namely, the element $(0, 1)$ is mapped to $(0, \frac{1}{\sqrt{2}})$ and $(1, \frac{1}{\sqrt{2}})$, and the element $(1, 0)$ is mapped to $(0, 0)$ and $(1, 0)$. *Oh no! Now we have two amplitudes for each bitstring! Our state is cooked! What should we do?*

We need to sum up the amplitudes pertaining to the same bitstring. Since on the right the amplitudes are $0$, we end up with the following state:



Let's now apply the Hadamard gate again to this superposition state:



Here things are a bit more interesting. The first element $(0, \frac{1}{\sqrt{2}})$ is mapped to $(0, \frac{1}{2})$ and $(1, \frac{1}{2})$. The second element $(1, \frac{1}{\sqrt{2}})$ is mapped to $(0, \frac{1}{2})$ and $(1, -\frac{1}{2})$. When we sum up the amplitudes, the $(1, \frac{1}{2})$ and $(1, -\frac{1}{2})$ cancel out, producing $(0, 0)$ and $(1, 0)$, leaving us with the original state:

$$\begin{matrix} 0 & 1 \\ \{1 & 0\} \end{matrix}$$

This example demonstrates the crucial notion of *destructive interference*, which you may recall from high-school physics when you studied waves. The two amplitudes for $1$ cancel out. (Also, this example shows that the Hadamard gate is its own inverse.)

Now, let's implement the Hadamard gate. Say we want to apply it to the $j$th qubit in a state $s$:

$$\begin{aligned} & s.\mathsf{flatMap}(\lambda b, a.\ \{ \\ & \quad (b^{j=0}, \frac{a}{\sqrt{2}}), \\ & \quad (b^{j=1}, \frac{a(1-2b_j)}{\sqrt{2}}) \\ & \}) \\ & .\mathsf{reduceByKey}(\lambda x, y.\ x+y) \end{aligned} \tag{$H_j$}$$

Let's break this expression down.

- The flatMap's lambda function takes a pair $(b, a)$ and returns a *set* containing two pairs. By definition, the flatMap takes the union of the generated sets, producing one big set.
- $b^{j=0}$ is $b$ but with the $j$th bit set to $0$, while $b^{j=1}$ is $b$ but with the $j$th bit set to $1$.
- The reduceByKey function then sums up the amplitudes associated with each bitstring (the keys).

## Universality

As we saw above, we used two simple templates to implement quantum gates, a map or a flatMap followed by a reduceByKey. You can implement any quantum operation using these two templates.[3]

Indeed, the set of gates we have seen above are *universal*, meaning that together they can be used to approximate any quantum mechanical operation to an arbitrary degree of accuracy (by stacking enough of them together). What's interesting is that if we remove any of the gates (except $X$), we lose the property of universality! For example, if we remove the $T$ gate, we are left with what are know as *Clifford gates*, which can be efficiently simulated on a classical computer [6]: If you have a program comprised of only Clifford gates, you

[3] *Exercise for the reader*: implement all of the quantum gates we introduced above in SQL.

don't need to use the above exponential encoding of a quantum state and instead you can use a polynomial one in the number of qubits. Once the $T$ gate is added, we have no way of simulating a quantum program efficiently on a classical computer—at least no way we know of!

# 4 The Mysterious Measurement

In the previous section, we saw how to manipulate quantum states using quantum gates. But how do we read the state of a quantum computer? Ideally, I should be able to just say "*give me the amplitude of* $01$" and get an answer. Sadly, this is not possible.

Instead, the laws of quantum mechanics give us "measurements". Say I want to measure the state of the qubit. I will get a random outcome, either $0$ or $1$, with probabilities proportional to the respective amplitudes. Specifically, consider the single-qubit state

$$\begin{matrix} 0 & 1 \\ \{a & b\} \end{matrix}$$

If I measure this qubit, I will get $0$ with probability $|a|^2$ and $1$ with probability $|b|^2$. Similarly, if I have a two-qubit state

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{\ a & b & c & d\ \} \end{matrix}$$

and measure the leftmost qubit, I will get $0$ with probability $|a|^2 + |b|^2$ (the amplitudes of the elements $0*$) and $1$ with probability $|c|^2+|d|^2$ (the amplitudes of the elements $1*$).

## Probability computation

Let's implement this probability computation operation. Say we want to compute the probability of the $j$th qubit being $0$ in a state $s$:

$$\begin{aligned} &s.\mathsf{filter}(\lambda b, a.\ b_j = 0) \\ &\ .\mathsf{map}(\lambda b, a.\ |a|^2) \\ &\ .\mathsf{sum}() \end{aligned} \qquad (p_{j,0})$$

The filter function keeps only the elements where the $j$th qubit is $0$. The map function takes each of those elements and squares their amplitude. The sum function sums up the squared amplitudes.

## Post-measurement state

What happens to the state after measuring a qubit? If you measure $0$ for the $j$th qubit, you throw away elements where the $j$th qubit is $1$ (i.e., you zero out their amplitudes). If you measure $1$, you throw away elements where the $j$th qubit is $0$.

The only wrinkle is if we zero out some amplitudes, the state is no longer normalized. Take the following state

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{\frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2}\} \end{matrix}$$

and say we measure the leftmost qubit and get $0$ (which we get with probability $1/2$). Then, we throw away the element $10$ and $11$ and are left with the following state:

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{\frac{1}{2} & \frac{1}{2} & 0 & 0\} \end{matrix}$$

This state is not a valid quantum state because $(1/2)^2 + (1/2)^2 = 1/2 \neq 1$. We fix this issue by dividing the amplitudes by the square root of the probability of the measurement outcome, i.e., $\sqrt{1/2}$, obtaining the following state, which is now normalized:

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 0\} \end{matrix}$$

Let's implement this post-measurement state update. Say we want to update the state $s$ after measuring the $j$th qubit and getting $0$ with probability $p$. The new post-measurement state is given by

$$s.\mathsf{map}(\lambda b, a.\ (b, (1 - b_j)ap^{-0.5})) \qquad (\text{measure}_{j,0})$$

where the $(1 - b_j)$ is used to zero out the amplitude if the $j$th qubit is $1$.

## Entanglement and Quantum Weirdness

When you start applying measurements, you will notice some funny things happening. For example, say we have the following state:

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{\frac{1}{\sqrt{2}} & 0 & 0 & \frac{1}{\sqrt{2}}\} \end{matrix}$$

If you measure the leftmost qubit, you will get $0$ with probability $1/2$. If you measure the rightmost qubit, you will get $1$ with probability $1/2$.

Now, say we measure the leftmost qubit and get $0$. The post-measurement state is the following (recall that we zero out the amplitudes of $1*$ and renormalize the remaining amplitudes):

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \{1 & 0 & 0 & 0\} \end{matrix}$$

So now if we measure the rightmost qubit, we will get $0$ with probability $1$.

The measurement of the left qubit affected the right qubit! At the beginning, the right qubit was in a superposition of $0$ and $1$ with equal probability. But after the measurement, the rightmost qubit "*collapsed*" to $0$. In quantum mechanics, we say that the two qubits are *entangled*.

Entanglement has wide-ranging practical and philosophical implications. Einstein famously called entanglement "spooky action at a distance," as it seems to violate locality—the idea that objects can only be influenced by their immediate surroundings. The fact that measuring one qubit can instantaneously affect another qubit, no matter how far apart they are, troubled Einstein and other physicists. This concern led to the famous EPR (Einstein-Podolsky-Rosen) paradox paper [4] questioning quantum mechanics. Indeed the above state is called an *EPR pair* (or a Bell pair).

The philosophical implications of measurement and entanglement continue to be debated today, leading to unsettling interpretations of quantum mechanics like the many-worlds interpretation—where each measurement leads to a new universe! Anyway, in the universe in which you are currently reading this paper, we will not go into the philosophical implications of quantum mechanics, but I hope that this example gives you a taste of the weirdness of quantum mechanics.

# 5 Some Quantum Algorithms

We are now ready put together the pieces we have discussed so far and implement some quantum algorithms.

## Constructing an EPR pair

We will start with the simple idea of creating an EPR pair, which is the following state:

$$\begin{matrix} 00 & 01 & 10 & 11 \\ \left\{\frac{1}{\sqrt{2}} & 0 & 0 & \frac{1}{\sqrt{2}}\right\} \end{matrix}$$

As is customary, we're going to start from the $00$ state—the *ground state*:

$$\begin{array}{cccc} 00 & 01 & 10 & 11 \\ \{1 & 0 & 0 & 0\} \end{array}$$

Because our goal is to arrive at some $1/\sqrt{2}$ amplitudes, it sounds like we need a Hadamard gate. Let's apply a Hadamard gate to the leftmost qubit. This operation produces the following state:

$$\begin{array}{cccc} 00 & 01 & 10 & 11 \\ \left\{\frac{1}{\sqrt{2}} & 0 & \frac{1}{\sqrt{2}} & 0\right\} \end{array}$$

This state is close to what we want, but we need to move the $(10, 1/\sqrt{2})$ element to $(11, 1/\sqrt{2})$. To do so, we can apply a $CX$ gate to the leftmost qubit as the control and the rightmost qubit as the target. Recall that the $CX$ gate flips the target qubit if the control qubit is 1. Therefore, the element $(10, 1/\sqrt{2})$ is mapped to $(11, 1/\sqrt{2})$, while the element $(00, 1/\sqrt{2})$ remains unchanged. And now we have our EPR pair!

## Evaluating Boolean functions

Say we have a Boolean function $f : \{0, 1\} \to \{0, 1\}$. How can we implement this function on a quantum computer? Remember, quantum operations are reversible, so we need to make this function reversible somehow. One simple way to do this is using two qubits, one for the input and one for the output of $f$. Let's say the system starts in the state $xy$, where $x$ is the input and $y$ is the output. We want to transform this state to $x(y \oplus f(x))$ via quantum operations, where $\oplus$ is the exclusive-or operator. In other words, the input bit $x$ is left unchanged, but the output bit $y$ is mapped to $y \oplus f(x)$. Convince yourself that this is a reversible operation by working out the truth table for the operation.

In our notation, we want to implement the following operation:

$$s.\mathsf{map}(\lambda b, a.\ (b_0(b_1 \oplus f(b_0)), a))$$

where $b_0$ is the input qubit and $b_1$ is the output qubit.

**Example A** Suppose we have the function $f(x) = 1$, which is non-reversible. We can implement this function as follows, using two qubits: Simply, apply $X$ to the output qubit. Let's write this operation explicitly as a map:

$$s.\mathsf{map}(\lambda b, a.\ (b^{\neg 1}, a)) \tag{$X_1$}$$

Note that $b^{\neg 1}$ is equivalent to $b_0(b_1 \oplus 1)$.

**Example B** Now let's try a slightly more complicated function, $f(x) = \neg x$. Therefore, we want to implement the following map:

$$s.\mathsf{map}(\lambda b, a.\ (b_0(b_1 \oplus \neg b_0), a))$$

We can implement this map as follows: (1) apply a $CX$ gate to the input and output qubits, and (2) apply an $X$ gate to the output qubit. Let's express this sequence of operations using our map notation:

$$\begin{aligned} &s.\mathsf{map}(\lambda b, a.\ (ite(b_0, b^{\neg 1}, b), a) && (CX_{0,1}) \\ &.\mathsf{map}(\lambda b, a.\ (b^{\neg 1}, a)) && (X_1) \end{aligned}$$

We can combine these into a single map:

$$s.\mathsf{map}(\lambda b, a.\ (ite(b_0, b, b^{\neg 1}), a))$$

You can verify that $ite(b_0, b, b^{\neg 1})$ is equivalent to $b_0(b_1 \oplus \neg b_0)$.

**Evaluation of $f(x)$** Now if want to evaluate an $f(x)$ on some input $x$, we can put $f$ in the above form and evaluate it starting with the state $x0$, which will give us the state $x(0 \oplus f(x)) = xf(x)$. We can then just measure the output qubit to get the value of $f(x)$.

## Parallel evaluation

Perhaps the coolest thing about quantum computing is evaluating functions in parallel on all possible inputs. Say we have encoded our Boolean function $f$ as described above. To evaluate $f$ on both inputs $0$ and $1$, we can start with the state $00$ and apply a Hadamard gate to the input qubit, resulting in the following state:

$$\begin{array}{cccc} 00 & 01 & 10 & 11 \\ \left\{ \frac{1}{\sqrt{2}} \right. & 0 & \frac{1}{\sqrt{2}} & \left. 0 \right\} \end{array}$$

Notice how the state is a superposition of the two possible inputs, $0$ and $1$. Now, applying the function $f$ to the input qubit (using the encoding we described above), we get the following state:

$$\begin{array}{cccc} 0f_0 & 0\bar{f}_0 & 1f_1 & 1\bar{f}_1 \\ \left\{ \frac{1}{\sqrt{2}} \right. & 0 & \frac{1}{\sqrt{2}} & \left. 0 \right\} \end{array}$$

where $f_x = f(x)$ and $\bar{f}_x = \neg f(x)$.

Notice how we have computed $f(0)$ and $f(1)$ in parallel, with one call to $f$, thanks to superposition. The challenge, of course, is that we cannot extract $f(0)$ and $f(1)$ from this state at will. Instead, if we measure the right qubit, and we will get $f(0)$ or $f(1)$ with probability $1/2$.

So while quantum parallelism may appear amazing, its benefits are far from immediate! The following algorithm gives a taste of how we can extract some results from parallel evaluation.

## Deutsch's algorithm

Deutsch's algorithm [3] is a cool little algorithm that demonstrates the advantages of quantum computing over classical computing. The algorithm takes a function $f : \{0, 1\} \to \{0, 1\}$ and determines if it is constant or balanced. A constant function maps all inputs to the same output, while a balanced function maps the two inputs to different outputs.

The classical algorithm for this problem takes two evaluations of $f$ to solve the problem. The quantum algorithm takes only one evaluation of $f$, and this generalizes to functions with $n$ inputs. Only one evaluation of $f$ is needed!

We've seen all the ingredients we need to implement this algorithm: (1) we encode $f$ as a reversible operation, and (2) evaluate $f$ in parallel on all possible inputs using quantum parallelism.

Our starting state is $00$, and we will use the left qubit as the input of $f$ and the right qubit as its output. First, we negate the output qubit, resulting in the state $01$. Then, we apply a Hadamard to *both* qubits—not just the input qubit as we did in the previous section. This results in the state:

$$\begin{array}{cccc} 00 & 01 & 10 & 11 \\ \{\frac{1}{2} & -\frac{1}{2} & \frac{1}{2} & -\frac{1}{2}\} \end{array}$$

Notice the negative amplitudes, resulting from the fact that the output qubit is negated. Throughout this algorithm, all the amplitudes will have the same magnitude, so we'll just represent them as "+" or "-" for conciseness.

$$\begin{array}{cccc} 00 & 01 & 10 & 11 \\ \{+ & - & + & -\} \end{array}$$

Now we apply $f$, which produces the following state:

$$\begin{array}{cccc} 0f_0 & 0\bar{f}_0 & 1f_1 & 1\bar{f}_1 \\ \{+ & - & + & -\} \end{array}$$

In other words, we have a superposition of the two possible outputs of $f$ and their negations! The question now is how can we use this state to determine if $f$ is constant or balanced? If we just measure the second qubit, we cannot deduce anything about $f$, because it will give us a random outcome.

The trick is to apply an operation that concentrates the amplitude on the leftmost qubit. If the function is constant, the amplitude will all be on $0*$; if the function is balanced, the amplitude will be on $1*$. We will exploit the fact that we have all the negative amplitudes to help cancel things out.

Let's apply a Hadamard gate to the first qubit and let's take a look at the resulting state before reduceByKey, just to get a feeling for the cancellations that will happen.

$$\begin{array}{cccccccc} 0f_0 & 1f_0 & 0\bar{f}_0 & 1\bar{f}_0 & 0f_1 & 1f_1 & 0\bar{f}_1 & 1\bar{f}_1 \\ \{+ & + & - & - & + & - & - & +\} \end{array}$$

Now, suppose that $f$ is constant. This means that $f(0) = f(1)$. In this case, all the amplitudes $1*$ will cancel out. For example, $(1\bar{f}_0, -)$ and $(1\bar{f}_1, +)$ will cancel out in the reduction. This reduction produces the following state:

$$\begin{array}{cccc} 0f_0 & 0\bar{f}_0 & 1f_1 & 1\bar{f}_1 \\ \{+ & - & 0 & 0\} \end{array}$$

Notice that all the amplitudes are concentrated on $0*$. So if we measure the first qubit, we will get $0$ with probability $1$.

Now, suppose that $f$ is balanced, which means that $f(0) \neq f(1)$. In this case, all the amplitudes $0*$ cancel out, leaving us with the state:

$$\begin{array}{cccc} 0f_0 & 0\bar{f}_0 & 1f_1 & 1\bar{f}_1 \\ \{0 & 0 & + & -\} \end{array}$$

So if we measure the first qubit, we will get $1$ with probability $1$.

## The Toffoli gate

In all of our examples so far, we did not use the $T$ or $S$ gates. Indeed, we only had to use real numbers for the amplitudes. So when do we need the $T$ or $S$ gates?

Consider the basic logical AND operation, $f(x, y) = x \wedge y$. To encode this function as a reversible operation, we can use the trick we learned earlier, where we use an extra qubit to store the output of the function. Basically, we want to map the bitstring $xyz$, where $z$ is the output, to

$$xy(z \oplus (x \wedge y))$$

This operation is known as the Toffoli gate.

It turns out, to implement the Toffoli gate, we need to use a $T$ gate! This is because $CX$ and $X$ alone are not enough to implement an AND. Specifically, we need a remarkably long sequence of $CX, H$, and $T$ gates (more than 10 gates!) just to implement this simple operation. I won't show the exact sequence here, because it isn't very instructive.

# 6 Implementation

I've implemented our quantum states and operations as a Python class, `State`, with just about 80 lines of code. In addition to holding the quantum state, as a set of pairs, it also maintains classical registers to hold the results of measurements if desired.

The implementation closely resembles our pseudocode above. Take, for instance, our $S$ operation,

$$s.\mathsf{map}(\lambda b, a.\ (b, ai^{b_j}))$$

In Python, this expression is implemented as follows:

```
self.state.smap(lambda b, a: (b, a*(1j**b[j])))
```

where `1j` is the imaginary unit and `smap` is the `map` function from the `pyfunctional` library.

**Example: EPR pair** One can easily implement quantum algorithms as a sequence of operations on a state. For example, here's how we can prepare an EPR pair:

```
State(2).h(0)
        .cx(0,1)
```

where `State(2)` is the initial state $00$, `h(0)` is the Hadamard on the first qubit, and `cx(0,1)` is the $CX$ on the first and second qubits.

**Example: Deutsch's algorithm** Similarly, we can implement Deutsch's algorithm as follows for the function $f(x) = 1$:

```
State(2).h(0)
        .x(1)
        .h(1)
        .x(1) # f(x) = 1
```

```
        .h(0)
        .measure(0)
```

**Example: Classical branching** Our implementation also supports classical branching. Some algorithms, like *quantum teleportation* [2] (a mind-bending algorithm that we don't cover here), use the results of measurements to decide which operations to apply. To give you a taste, here's how we can implement classical branching:

```
s = State(2,1)
s.h(0)
 .cx(0,1)
 .measure(1,0)
if s.cbits[0]:
   s.cx(0,1)
else:
   s.x(1)
```

We started by creating a state with 2 qubits and 1 classical bit. Then, we stored the result of measurement in the classical bit, `0`, as indicated by the `measure(1,0)` call. Then, we used the value of the classical bit (`s.cbits[0]`) to decide which operation to apply, $CX$ or $X$.

# 7 Notes

I've presented the basics of quantum computing through the lens of set manipulation using standard programming constructs. It turns out that this approach is a very natural way to think about quantum computing, and it is very easy to implement. The view is similar in spirit to the path integral/sum formulation of quantum mechanics [5]. Indeed, most presentations of quantum computing (e.g., Nielsen and Chuang [7]) tend to use a similar formulation at certain points of exposition, e.g., saying that a $CX$ is a map $|xy\rangle \mapsto |xy \oplus f(x)\rangle$, but they tend to stop there and continue with a linear algebraic view. For a formal treatment of quantum computation using the path sum formulation, see Amy [1].

While my focus here is on a programmatic and simple introduction to quantum computing, I believe that this view may be a good foundation for automated reasoning about quantum programs. For example, we can compactly encode the map-style operations of a quantum program in a fragment of first-order logic, and then use SMT solvers to check if the program is correct.

## Acknowledgments

I would like to thank Tom Reps, Subhajit Roy, Amanda Xu, Abtin Molavi, Swamit Tannu, Charles Yuan, Tancrède Lepoint, and Toby Murray for their comments. I would also like to thank Cursor and its army of models for help with the figures.

# References

[1] Matthew Amy. Towards large-scale functional verification of universal quantum circuits. *Quantum Programming Languages and Systems (QPL'18)*, 2018. URL https://arxiv.org/abs/1805.06908.

[2] Charles H. Bennett, Gilles Brassard, Claude Crépeau, Richard Jozsa, Asher Peres, and William K. Wootters. Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels. *Physical Review Letters*, 70(13):1895–1899, 1993. doi: 10.1103/PhysRevLett.70.1895.

[3] David Deutsch. Quantum theory, the Church-Turing principle and the universal quantum computer. *Proceedings of the Royal Society of London A*, 400 (1818):97–117, 1985. doi: 10.1098/rspa.1985.0070.

[4] Albert Einstein, Boris Podolsky, and Nathan Rosen. Can quantum-mechanical description of physical reality be considered complete? *Physical Review*, 47(10):777, 1935. doi: 10.1103/PhysRev.47.777.

[5] Richard P. Feynman. Space-time approach to non-relativistic quantum mechanics. *Reviews of Modern Physics*, 20(2):367–387, 1948. doi: 10.1103/RevModPhys.20.367.

[6] Daniel Gottesman. *The Heisenberg Representation of Quantum Computers*. PhD thesis, California Institute of Technology, Pasadena, CA, 1998.

[7] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010. ISBN 978-1-107-00217-3.